CS 514 · TEAM PROJECT · TECHNICAL REPORT

Baseball Attendance Simulator

Predicting MLB game attendance and secondary-market ticket prices from 35 seasons of game logs, weather, scheduling, and 2026 ticket listings

Team	Grant Trager · Kyle Kline · Rodolfo Baza Carrillo · Jimmy Ly ·
Date	October 13 2026
Application repo	github.com/BazaGazza/BaseBallAttendanceSimulator
Website repo	github.com/BazaGazza/BaseballWebsite
Stack	Python 3 · SQLite · pandas · Dash/Plotly · Astro

This report covers the design, implementation, database and course concepts applied, screenshots of the working system, and a user guide for installing and running every component.

Contents

1 Project Overview

2 System Design

Architecture · Data sources · Relational schema · Views · Indexes

3 Implementation

ETL pipeline · Query layer · Prediction engine · CLI · Dashboard · Website

4 Course Concepts Applied

5 Results and Screenshots

6 Query Performance Analysis

7 User Guide

8 Limitations and Future Work

Team Contributions

1 Project Overview

Problem

Teams set ticket prices and plan promotions before they know how many fans will show up. Attendance depends on many things at once: who the opponent is, what day it is, whether the game is in the afternoon or evening, and the weather. Public data on all of these exists, but it is spread across sources that do not share identifiers, which makes it hard to ask a simple question like *"How many people should we expect for a Friday night Dodgers game in San Diego, and what are tickets going for?"*

Solution

The Baseball Attendance Simulator (BAS) merges historical MLB game logs (Retrosheet, 1990–2025) with current secondary-market ticket listings (Gametime and SeatGeek, 2026 season) into a single normalized SQLite database. On top of that database it provides:

- **An attendance simulator** (command line) that predicts attendance for a hypothetical game, explains which factors raise or lower it, and estimates ticket prices for the matchup.
- **An analytics dashboard** (Python Dash) with league-wide summary statistics and interactive charts.
- **A query-performance benchmark** that measures the effect of the composite indexes.
- **A project website** (Astro) that explains the motivation, team, and usage to a general audience.

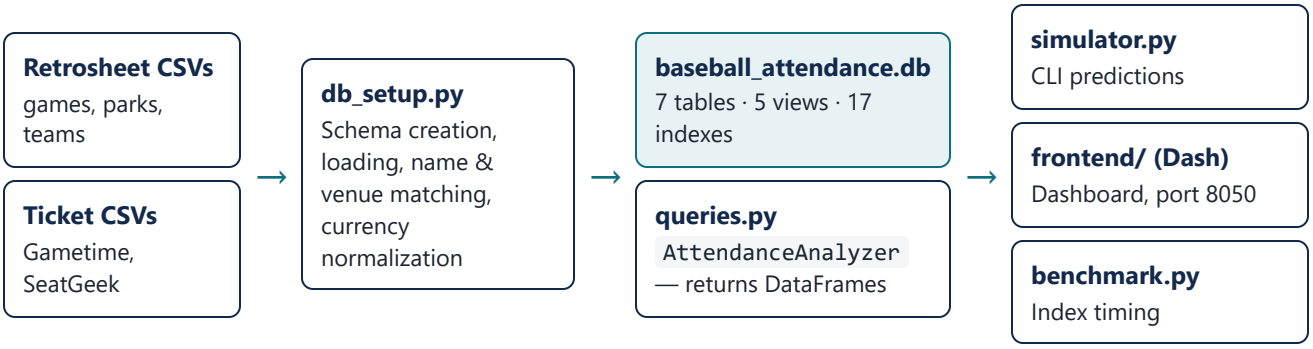
Scope at a glance

Item	Count	Notes
Games	84,430	1990–2025; 83,219 regular-season games
Ballparks	672	Every park in the Retrosheet registry; 42 used since 2020
Team codes	36	Includes historical franchises (e.g. MON, FLO)
Gametime ticket events	2,437	2026 season, prices stored in cents
SeatGeek ticket events	2,285	2026 season, prices stored in dollars
Unified ticket pricing rows	4,722	Via the `v_ticket_pricing` view

2 System Design

2.1 Architecture

The system is layered so that each part only depends on the one beneath it. All SQL lives in the database layer and the query layer; the user-facing tools never build SQL themselves.



The Astro website is a separate static site that documents the project and links to the application repository; it does not query the database.

2.2 Data sources

File	Source	One row is…	Key
`clean_games.csv`	Retrosheet game logs	One completed game: date, park, teams, attendance, weather, score	`game_id`
`clean_parks.csv`	Retrosheet park registry	One ballpark	`park_id`
`clean_teams.csv`	Retrosheet	One franchise code	`team_id`
`clean_gametime.csv`	Gametime marketplace	One 2026 game listing with min/max price and trending score	`eventId`
`clean_seatgeek.csv`	SeatGeek marketplace	One 2026 game listing with low/avg/high price, listing and ticket counts	`eventId`

The two ticket sources do not use Retrosheet codes. They identify games by a text name such as `"Pittsburgh Pirates at New York Mets (Opening Day)"` and venues by their own internal IDs. Linking them to the historical data is the main integration problem the design solves (Section 3.1).

2.3 Relational schema

PK marks primary keys and **FK** marks foreign keys. Foreign-key enforcement is turned on with `PRAGMA foreign_keys = ON`.

parks · 672 rows
park_id PK
park_name, aka
city, state
start_date, end_date
league, notes

teams · 36 rows
team_id PK

team_lookup · 30 rows
team_id PK FK→teams
full_name
city

games · 84,430 rows
game_id PK
game_date, season, day_of_week
park_id FK→parks
home_team_id FK→teams
vis_team_id FK→teams
attendance, daynight
temp, sky, precip
wind_dir, wind_speed, field_cond
home_runs, vis_runs, total_runs
gametype

gametime_events · 2,437
event_id PK
event_name, event_date
park_id FK→parks
home_team_id FK→teams
vis_team_id FK→teams
min_price, max_price (cents)
trending_score, source_venue_id

seatgeek_events · 2,285
event_id PK
event_name, event_date
park_id FK→parks
home_team_id FK→teams
vis_team_id FK→teams
lowest/avg/highest_price ($)
event_score, popularity_score
listing_count, ticket_count

venue_map · 30 rows
park_id PK FK→parks
gt_venue_id UNIQUE
sg_venue_id UNIQUE
venue_name

Design notes. `teams` is the hub of the schema: every table that names a team references it, so a team code means the same thing everywhere. `team_lookup` and `venue_map` are bridge tables that translate the ticket sites' names and venue IDs into Retrosheet keys. Keeping them as tables (instead of hard-coding the mapping in queries) means a new data source only needs new rows, not new SQL. The UNIQUE constraints on the two venue IDs guarantee the mapping is one-to-one.

2.4 Views

Views hold the analytical logic, so the simulator and dashboard read pre-defined result sets instead of repeating aggregate SQL.

View	Purpose	Technique
`v_ticket_pricing`	Combines both ticket sources into one table with common column names and dollar units	`UNION ALL`, column aliasing, unit conversion (÷100)
`v_team_attendance_stats`	Per-team, per-season games, average, min, max attendance	`GROUP BY` with aggregates
`v_matchup_history`	Attendance for every home/visitor pairing	`GROUP BY` on two keys
`v_day_of_week_effects`	How far each weekday moves a team's attendance from its own average	Two CTEs joined on team
`v_weather_impact`	Same comparison for sky × precipitation × temperature bucket	CTEs, `CASE` bucketing, join

2.5 Indexes

Single-column indexes cover the common filters (`season`, `home_team_id`, `park_id`, `game_date`, `daynight`, `gametype`, and the event date/team/park columns on both ticket tables). Five composite indexes were added specifically for the simulator's multi-column lookups:

```
CREATE INDEX idx_games_team_dow      ON games(home_team_id, day_of_week);
CREATE INDEX idx_games_team_daynight ON games(home_team_id, daynight);
CREATE INDEX idx_games_team_weather  ON games(home_team_id, sky, precip);
CREATE INDEX idx_gt_team_matchup     ON gametime_events(home_team_id, vis_team_id);
CREATE INDEX idx_sg_team_matchup     ON seatgeek_events(home_team_id, vis_team_id);
```

Each one leads with `home_team_id` because every simulator query filters on the home team first. Their measured effect is in Section 6.

3 Implementation

3.1 ETL pipeline — `db_setup.py`

The setup script builds the database from scratch in one run. If the database already exists it stops unless `--force` is passed, so it cannot overwrite data by accident.

1. **Validate inputs.** Confirms all five CSVs exist before creating anything.
2. **Create the schema.** Runs the full DDL script: tables, constraints, indexes, and views.
3. **Load historical data.** Reads parks, teams, then games with pandas (parents before children so foreign keys are satisfied) and converts pandas missing values to SQL `NULL`.
4. **Load bridge tables.** Inserts the 30 current teams into `team_lookup` and the 30 ballparks into `venue_map`, adding any team codes not already present.
5. **Match ticket events to teams.** A regular expression splits event names into visitor and home team:

   ```
   _NAME_PATTERN = re.compile(r"^(.+?)\s+at\s+(.+?)(?:\s*\(.*)?$")
   # "Pittsburgh Pirates at New York Mets (Opening Day)"  ->  ("PIT", "NYN")
   ```

   Each name is looked up in `team_lookup` (exact match first, then a prefix match so suffixes like `" - Opening Day"` still resolve). Events that cannot be matched are counted and skipped rather than stored with bad keys. In the current data every event matched.
6. **Match venues.** Each event's venue ID is translated to a Retrosheet `park_id` through `venue_map`.
7. **Commit and report.** Commits once at the end and prints row counts for every table and the pricing view.

3.2 Query layer — `queries.py`

`AttendanceAnalyzer` is the only class that talks to the database. Each method opens a connection, runs one parameterized query, and returns a pandas DataFrame.

Method	Answers
`attendance_by_team(season)`	Average, total, min, max attendance by team and season
`attendance_by_day_of_week()`	Average attendance and runs per weekday
`attendance_day_vs_night()`	Day games vs. night games
`attendance_by_weather()`	Attendance by sky and precipitation
`attendance_by_temp_range()`	Attendance and scoring in 10°F bands
`park_run_factors(min_games)`	Runs at each park relative to league average (100 = neutral)
`attendance_trend()`	League average attendance by season
`top_matchups(top_n)`	Matchups with the highest average crowds
`simulate_attendance(...)`	Attendance prediction (below)
`factor_impacts(...)`	Fans added or lost from weekday and weather
`predict_ticket_prices(...)`	Floor, average, ceiling price for a matchup

3.3 Prediction engine

Attendance — a fallback ladder

The most specific estimate is the average of past games that match every condition, but rare matchups may not have enough of those. The simulator therefore tries progressively broader filters and uses the first one with enough history. It reports which level it used and the sample size, so the user can judge confidence.

Level	Filter	Min. games
1 · Exact Match	home team + visitor + weekday + day/night	3
2 · Matchup + Day of Week	home team + visitor + weekday	3
3 · Matchup Only	home team + visitor	5
4 · Team Baseline	home team only	—

Each level returns the mean (the prediction) plus the minimum and maximum (the expected range). The minimum-sample threshold is applied in SQL with `HAVING COUNT(*) >= n`, so an under-sampled level returns no row and the next level runs.

Factor impacts

Reads `v_day_of_week_effects` and `v_weather_impact` for the home team to show how many fans the chosen weekday and weather add or subtract compared with that team's overall average. Temperature is bucketed as Cold (<60°F), Moderate (60–85°F), or Hot (>85°F) to match the view.

Ticket prices

Averages floor, average, and ceiling prices from `v_ticket_pricing` for the exact matchup across both marketplaces. If neither site lists that matchup, it falls back to the home team's average across all opponents.

3.4 Command-line simulator — `simulator.py`

Runs in two modes. **Flag mode** takes every input as an argument and prints one report. **Interactive mode** (`-i`) prompts for each value, validates it (team IDs are checked against the `teams` table, a team cannot play itself, weekday and day/night must be valid), prints the report, and loops until Ctrl+C. Errors in a single simulation are caught and reported without ending the session.

3.5 Analytics dashboard — `frontend/`

A multi-page Python Dash application. `app.py` builds the header and a navigation bar generated from Dash's page registry, so a new file in `pages/` appears in the menu automatically. `components/cards.py` provides reusable stat cards and chart cards with loading spinners, and three CSS files in `assets/` apply a dark theme. The Overview page imports `AttendanceAnalyzer` directly, so the dashboard and the simulator always show the same numbers.

3.6 Project website — BaseballWebsite

A static site built with Astro 7 from its blog starter, extended with custom styling, a home page, an About page with statements from each team member, and blog posts written in Markdown (a Moneyball-inspired background piece and a user manual). Posts live in a typed content collection, so each post's title, description, date, and hero image are checked at build time. The site also generates an RSS feed and sitemap.

3.7 Benchmark — `benchmark.py`

Runs four representative simulator queries 100 times each with the composite indexes in place, drops the five composite indexes, runs them again, then recreates the indexes so the application is left unchanged.

4 Course Concepts Applied

Concept	Where and how it is used
Relational modeling	Entities (parks, teams, games, ticket events) identified from raw files and modeled as separate tables with a defined grain: one row per game, per park, per listing.
Normalization	Team and park details are stored once and referenced by key. Team names live only in `team_lookup`; venue IDs live only in `venue_map`. No table repeats another's descriptive attributes.
Primary and foreign keys	Every table has a primary key; games and both event tables reference `parks` and `teams`. `PRAGMA foreign_keys = ON` enforces them, and loading order (parents first) respects them.
Integrity constraints	`NOT NULL` on required game fields; `UNIQUE` on marketplace venue IDs; unmatched ticket events rejected instead of stored with invalid keys.
Data cleaning & integration	Retrosheet "unknown" and sentinel values converted to `NULL`; entity resolution of free-text event names to team keys with regex and a lookup table; currency normalization (cents → dollars).
Joins	`games ⋈ parks` for park run factors; view-internal joins of per-team baselines against per-weekday and per-weather averages.
Aggregation	`COUNT`, `AVG`, `MIN`, `MAX`, `SUM` with `GROUP BY` throughout; `HAVING` to enforce minimum sample sizes.
Subqueries & CTEs	`WITH` clauses compute team baselines and league averages once, then join them to detail aggregates (`v_day_of_week_effects`, `v_weather_impact`, `park_run_factors`).
Set operations	`UNION ALL` merges two differently shaped sources into one pricing view, with `NULL` placeholders for columns only one source has.
Views	Five views encapsulate analytical logic so application code stays simple and consistent.
Indexing & query performance	Single and composite B-tree indexes chosen to match query filter order; measured before/after with a reproducible benchmark (Section 6).
Secure query construction	All user-supplied values are passed as `?` parameters, never concatenated into SQL, which prevents SQL injection from the interactive prompts.
Application layering	The database, a data-access class, and multiple front ends (CLI, web dashboard) are separated so each can change independently.

5 Results and Screenshots

5.1 Analytics dashboard

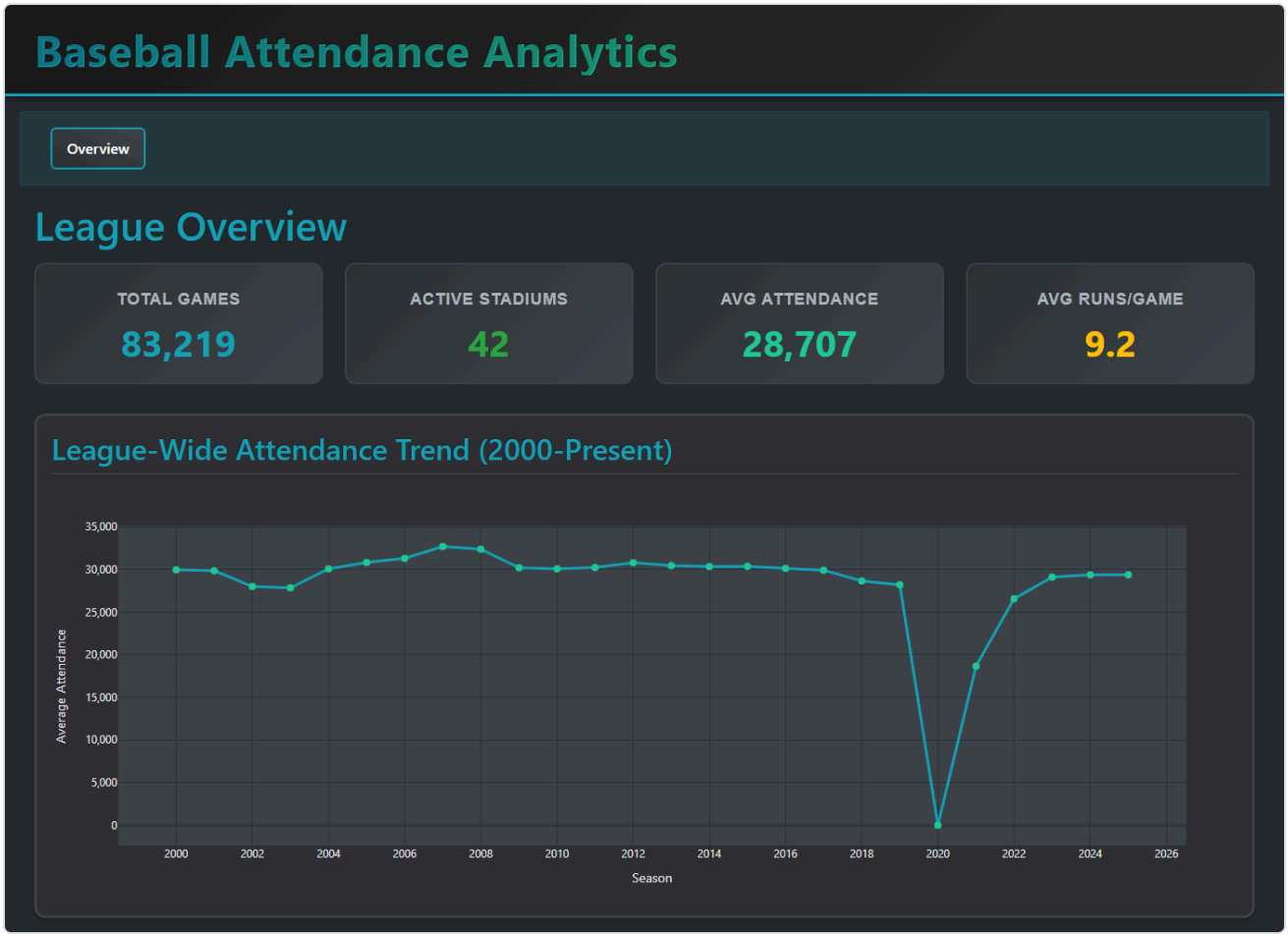


Figure 1. Dashboard Overview page (`http://localhost:8050`): headline statistics and the league-wide attendance trend. The 2020 drop reflects the season played without fans.

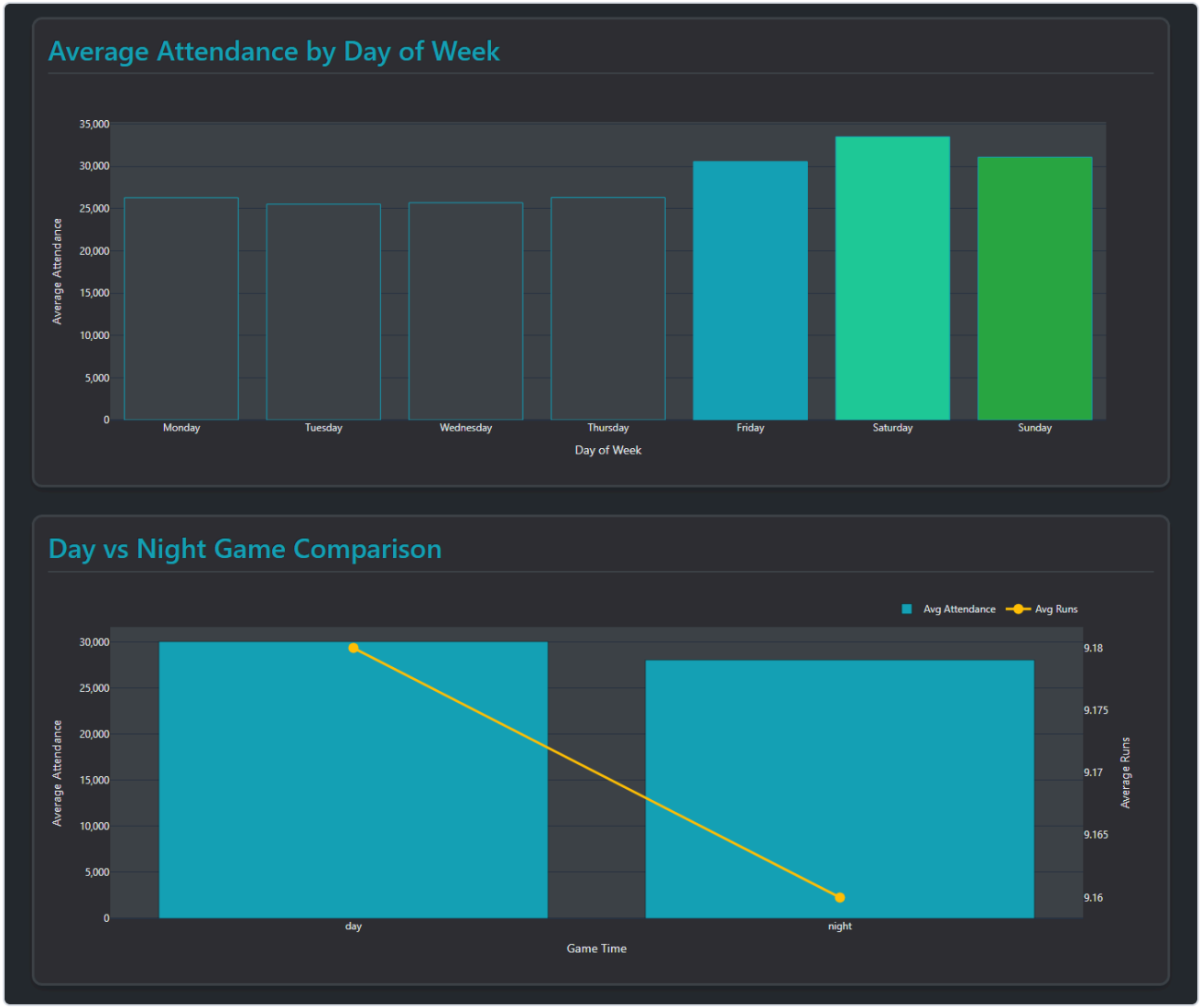


Figure 2. Average attendance by weekday (weekend highlighted) and day vs. night games, with average runs on the secondary axis.

5.2 Key findings from the data

Attendance by weekday

Day	Avg. attendance	Games
Saturday	33,499	13,531
Sunday	31,083	13,412
Friday	30,553	13,204
Thursday	26,316	9,140
Monday	26,290	8,728
Wednesday	25,692	12,648
Tuesday	25,520	12,556

Saturday averages about 8,000 more fans than Tuesday, about 31% more.

Attendance by temperature

Range	Avg. attendance	Avg. runs
< 50°F	25,519	8.88
50–59°F	27,127	8.74
60–69°F	28,935	8.82
70–79°F	28,226	9.10
80–89°F	29,830	9.55
90°F+	31,040	10.22

Warmer games draw larger crowds and more scoring; much of this is seasonal (summer weekends).

Biggest crowds by matchup (≥10 games)

Home vs. visitor	Avg.	Games
Dodgers vs. Yankees	53,970	17
Dodgers vs. Rays	48,991	10
Dodgers vs. Red Sox	48,384	12
Dodgers vs. Angels	48,359	77
Yankees vs. Mets	47,297	78

Park run factors (≥500 games)

Park	Factor
Coors Field (Denver)	129.7
Rangers Ballpark (Arlington)	117.5
Fenway Park (Boston)	109.1
Citi Field (New York)	88.5
Petco Park (San Diego)	85.7

100 = league average (9.17 runs/game). Coors Field produces ~30% more runs.

Day games average 30,035 fans against 28,046 for night games, about 2,000 more, largely because weekend afternoon games are included in the day group.

5.3 Simulator output

A real interactive session predicting a Friday night Dodgers game at Petco Park (user input in green):

```
PS> py -3.13 simulator.py -i
Entering Interactive Simulator Mode.
Press Ctrl+C to exit at any time.

Home Team ID (e.g. BOS): SDN
Visiting Team ID (e.g. NYA): LAN
Day of Week (e.g. Saturday): Friday
Day or Night (day/night): night
Include weather factors? (y/n): y
Sky (e.g. sunny, cloudy, overcast): sunny
Precipitation (e.g. none, rain, drizzle): none
Temperature (F): 72

============================================================
  BASEBALL ATTENDANCE SIMULATOR
============================================================
  Matchup: LAN at SDN
  Timing:  Friday (night)
  Weather: sunny, none, 72°F
------------------------------------------------------------

1. ATTENDANCE PREDICTION
  Predicted Attendance: 36,104
  Expected Range:       15,250 to 51,217
  Confidence Basis:     Exact Match (n=46)

2. FACTOR IMPACT ANALYSIS
  Day of Week (Friday)     : +2,461 fans
  Weather (sunny, none, Moderate): +314 fans

3. SECONDARY TICKET MARKET PREDICTION
  Predicted Floor Price:   $81.00
  Predicted Average Price: $200.00
  Predicted Ceiling Price: $2,207.00
  Data Source:             Exact Matchup (n=12)
============================================================
```

Reading the result: the estimate is based on 46 past Friday night Padres–Dodgers games, so it reached the most specific level of the fallback ladder. The wide range reflects the 1990s games at the Padres' previous stadium as well as sell-outs at Petco Park. Ticket estimates come from 12 listings for this matchup across both marketplaces.

5.4 Project website

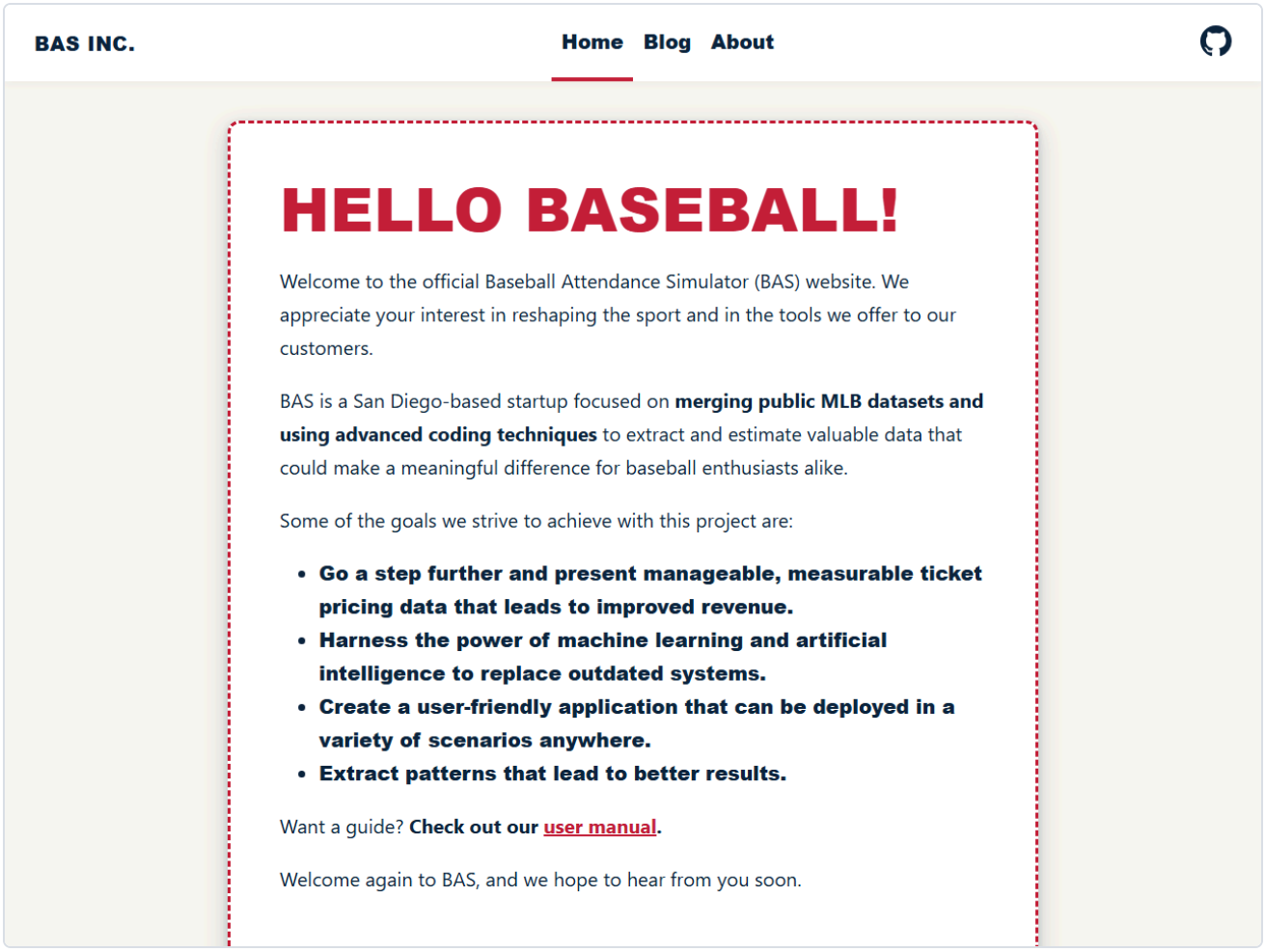


Figure 3. Website home page introducing BAS and its goals.

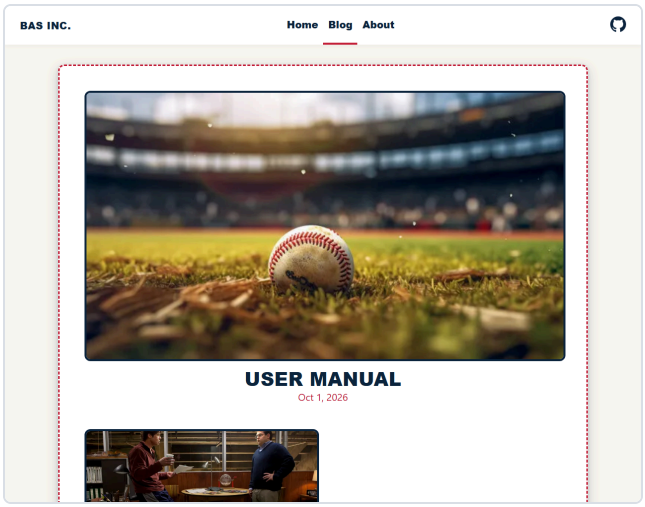


Figure 4. Blog index with the User Manual and Moneyball posts.

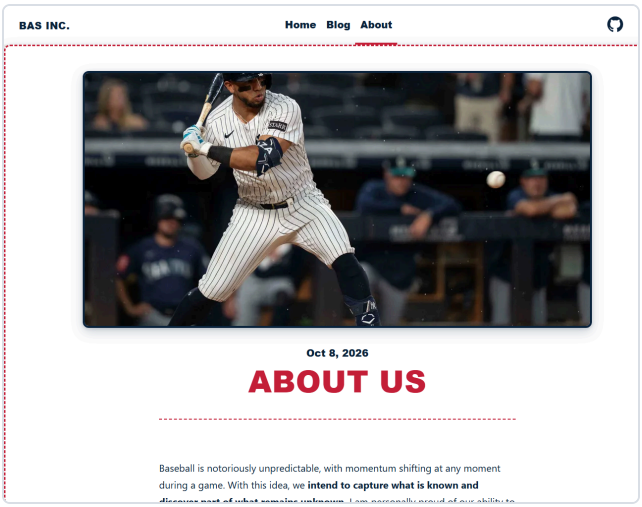


Figure 5. About page with statements from the team.

6 Query Performance Analysis

Each query was run 100 times and the total time recorded, first with all indexes and then with the five composite indexes dropped. The team's recorded results (Surface Pro, 16 GB RAM) were reproduced on a second Windows 11 machine for this report.

Query	Recorded run (ms / 100 runs)			Reproduction (ms / 100 runs)		
	No index	Indexed	Speedup	No index	Indexed	Speedup
Matchup prediction	1,104.59	297.14	**3.7×**	932.83	285.80	**3.3×**
Weather impact (view)	2,721.26	1,958.79	1.4×	2,402.23	1,816.19	1.3×
Day-of-week impact (view)	2,650.00	1,972.89	1.3×	2,347.79	1,705.79	1.4×
Ticket pricing (view)	5.61	3.19	1.8×	4.09	2.76	1.5×

Interpretation

- **Matchup prediction gains the most (3.3–3.7×).** Its filter on home team, visitor, weekday, and day/night is highly selective, and `idx_games_team_dow` / `idx_games_team_daynight` let SQLite jump to one team's games instead of scanning all 84,430 rows.
- **The two impact views gain less (1.3–1.4×).** Their CTEs compute a baseline over *all* of a team's games before filtering, so much of the work is aggregation that an index cannot skip. Materializing these baselines in a summary table would be the next optimization.
- **Ticket pricing is already fast** (~3 ms per 100 runs) because the event tables are small; the composite index still helps the `UNION ALL` branches.
- The two runs agree closely in their ratios, which shows the improvement comes from the indexes and not from one machine.

7 User Guide

7.1 Requirements

- Python 3.10 or newer with `pandas` (the simulator and database).
- For the dashboard: `dash`, `dash-bootstrap-components`, `dash-mantine-components`, `plotly` (listed in `frontend/requirements.txt`).
- For the website: Node.js 22.12 or newer.

> **Windows tip.** If more than one Python is installed, use the `py` launcher with a version (for example `py -3.13`) so you run the Python that has pandas installed. Plain `python` may point to a different installation.

7.2 Install and build the database

```
git clone https://github.com/BazaGazza/BaseBallAttendanceSimulator.git
cd BaseBallAttendanceSimulator
py -3.13 -m pip install pandas            # macOS/Linux: python3 -m pip install pandas
py -3.13 db_setup.py                      # creates baseball_attendance.db
```

To rebuild after changing the CSVs, run `py -3.13 db_setup.py --force`. This deletes and recreates the database file.

7.3 Run the simulator

Interactive mode (prompts for each value)

```
py -3.13 simulator.py -i
```

Answer each prompt and press Enter. After the report prints, the simulator asks for the next game. Press **Ctrl+C** to quit.

Prompt	Accepted values
Home / Visiting Team ID	A team code from the table below (not case-sensitive)
Day of Week	`Monday` … `Sunday`
Day or Night	`day` or `night`
Include weather factors?	`y` or `n`
Sky	`sunny`, `cloudy`, `overcast`, `dome`
Precipitation	`none`, `drizzle`, `rain`, `showers`, `snow`
Temperature (F)	A whole number, e.g. `72`

Single run with arguments

```
py -3.13 simulator.py --home BOS --vis NYA --dow Saturday --dn day --sky sunny --precip none --temp 75
```

`--home`, `--vis`, `--dow`, and `--dn` are required; `--sky`, `--precip`, and `--temp` are optional and only used together.

Team codes

Codes follow Retrosheet, which marks the league in two-team cities (e.g. `NYA` Yankees / `NYN` Mets, `CHA` White Sox / `CHN` Cubs).

Code	Team	Code	Team	Code	Team
ARI	Diamondbacks	HOU	Astros	PHI	Phillies
ATL	Braves	KCA	Royals	PIT	Pirates
BAL	Orioles	ANA	Angels	SDN	Padres
BOS	Red Sox	LAN	Dodgers	SFN	Giants
CHN	Cubs	MIA	Marlins	SEA	Mariners
CHA	White Sox	MIL	Brewers	SLN	Cardinals
CIN	Reds	MIN	Twins	TBA	Rays
CLE	Guardians	NYN	Mets	TEX	Rangers
COL	Rockies	NYA	Yankees	TOR	Blue Jays
DET	Tigers	ATH	Athletics	WAS	Nationals

Reading the report

- **Predicted Attendance** is the average of comparable past games; **Expected Range** is the lowest and highest of those games.
- **Confidence Basis** shows which fallback level was used and how many games (*n*) it is based on. "Exact Match" with a large *n* is the most reliable.
- **Factor Impact** values are fans above (+) or below (−) the home team's overall average.
- **Ticket prices** are averages across Gametime and SeatGeek 2026 listings. "Team Average" means no listing for that exact matchup was found.

7.4 Run the dashboard

```
py -3.13 -m pip install -r frontend/requirements.txt
cd frontend/src
py -3.13 app.py
```

Open **http://localhost:8050** in a browser. Stop the server with Ctrl+C. The database must be built first.

7.5 Run the benchmark

```
py -3.13 benchmark.py
```

Prints timings with and without the composite indexes, then restores the indexes.

7.6 Run the website

```
git clone https://github.com/BazaGazza/BaseballWebsite.git
cd BaseballWebsite
npm install
npm run dev            # http://localhost:4321
npm run build          # static site written to dist/
```

7.7 Troubleshooting

Symptom	Fix
`ModuleNotFoundError: No module named 'pandas'`	Install pandas for the Python you are running, or switch interpreters with `py -3.13`.
`FileNotFoundError: Database not found`	Run `db_setup.py` from the project root first.
`Database already exists`	Expected; use `--force` only if you want to rebuild.
`[!] invalid home team`	Use the Retrosheet code (e.g. `SDN`, not `SD`).
`UnicodeEncodeError` when saving output to a file	Set `$env:PYTHONIOENCODING='utf-8'` (PowerShell) before running.
`./run.sh` fails on Windows	The script expects a Unix virtual environment; use the commands in 7.4 instead.

8 Limitations and Future Work

Known limitations

- **2020 season.** Games played without fans are stored with attendance 0 (938 games). They are included in averages, which lowers team baselines slightly. Excluding `season = 2020` or storing these as `NULL` would remove the bias.
- **Weather is mixed up with time of day.** Night games are recorded with sky = `night`, so "sunny" only ever describes day games. A sunny-weather impact therefore partly measures the day-game effect.
- **Temperature sentinels.** 2,226 games still have `temp = 0` (mostly domed stadiums). These fall into the "Cold" bucket in `v_weather_impact`. Converting them to `NULL` would fix it.
- **Ticket ceiling prices.** 1,071 Gametime listings report a maximum price of 0, which lowers the average ceiling price.
- **Era and stadium changes.** Exact-match averages span 1990–2025, so they include old stadiums with different capacities (e.g. the Padres before Petco Park). This widens the expected range.
- **Ticket data is a snapshot** of the 2026 season, so prices do not change over time in the model.

Future work

- Weight recent seasons more heavily, or limit predictions to games at the team's current ballpark.
- Fit a regression model on weekday, time, weather, and opponent instead of averaging matching games.
- Add the planned dashboard pages: Team Performance, Weather Impact, Ballpark Explorer, and Ticket Pricing, including an interactive version of the simulator.
- Pull ticket prices on a schedule to study how prices move as game day approaches.
- Precompute team baselines in summary tables to speed up the impact views.

Team Contributions

Member	Role	Contributions
Grant Trager	Data & database	Collected and cleaned the MLB and ticket data, wrote the data dictionary, designed the SQLite schema, keys, views, and indexes, and wrote the query layer, simulator, and benchmark.
Kyle Kline	Dashboard	Built the Dash application, reusable components, theme, and the Overview page connected to the query layer.
Jimmy Ly	Website & support	Website administration and content, design support, and coordination across the team.
Rodolfo Baza Carrillo	Website	Built and deployed the Astro project website, wrote site content including the Moneyball background post, and linked the site to the application repository.